



Microteaching Unidad 7

Fundamentos, Enfoques y Niveles de Pruebas

Nuestro Recorrido de Aprendizaje

01

Por qué probamos

La mentalidad del tester y el propósito del testing profesional

03

Cómo probamos

Técnicas de diseño de pruebas: EP, BVA y Tablas de Decisión

02

Desde dónde probamos

Los 3 Enfoques fundamentales: Caja Negra, Blanca y Gris

04

Cuándo probamos

Los Niveles de Testing y la Pirámide del Testing

¿Por qué probamos el software?



El Testing NO es "buscar errores"

Muchos creen que el testing es simplemente encontrar bugs, pero el objetivo profesional va mucho más allá. El testing moderno se trata de garantizar valor y confianza en el producto.

El objetivo profesional verdadero es:

- **Medir la Calidad:** Obtener información objetiva y métricas sobre el estado real del software
- **Reducir el Riesgo:** Identificar problemas ANTES que lleguen al cliente final
- **Confirmar Requisitos:** Asegurar que construimos exactamente lo que se pidió

La Regla de Oro: El Costo Exponencial de un Bug

Invertir \$100 hoy nos ahorra \$10.000 mañana

En Requisitos

Corregir cuesta **\$1**

El momento más económico para detectar errores. Un malentendido en esta fase es fácil de corregir.

En Desarrollo

Corregir cuesta **\$100**

El código ya está escrito. Requiere refactorización y pruebas adicionales.

En Pruebas (QA)

Corregir cuesta **\$1.000**

Involucra múltiples equipos, documentación y re-testing completo.

En Producción

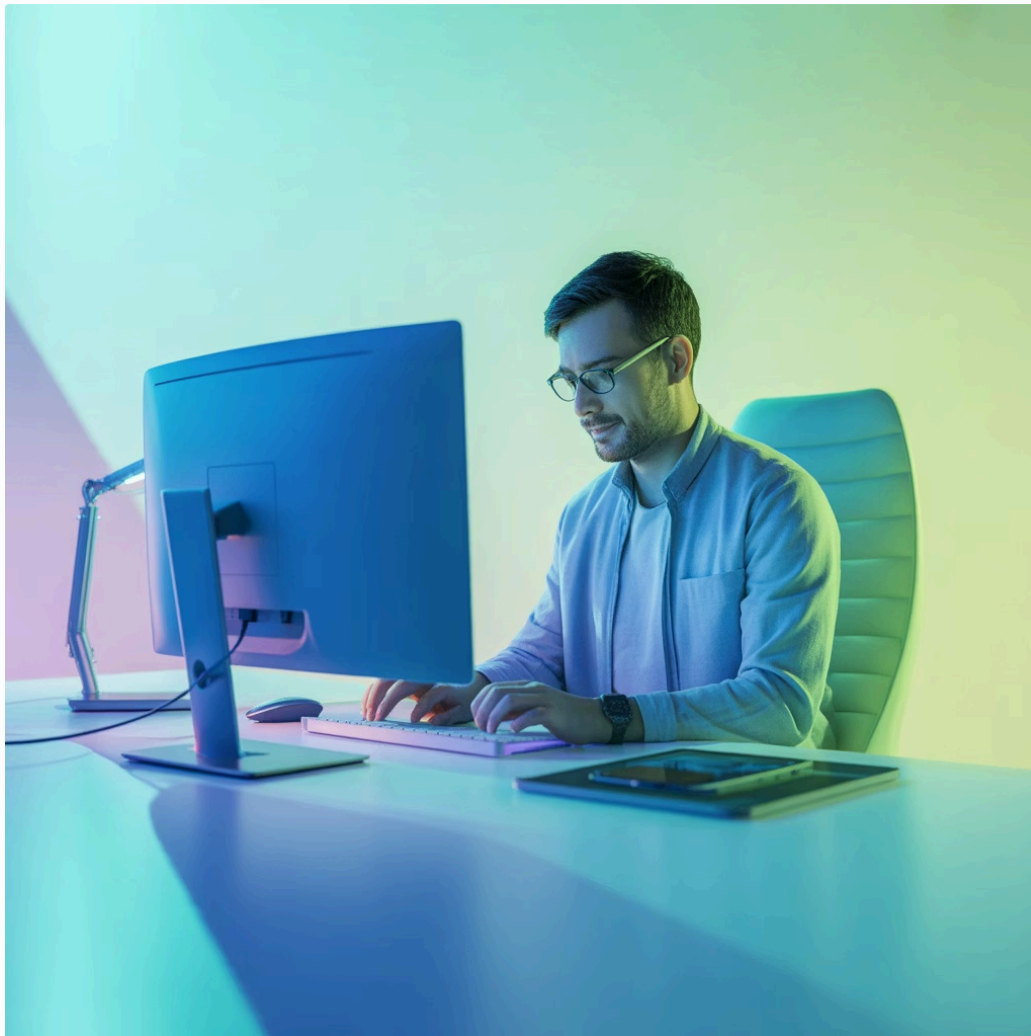
Corregir cuesta **\$10.000+**

Más la reputación, clientes perdidos, soporte técnico y posibles consecuencias legales.

☐ **Conclusión clave:** El testing no es un gasto, es una inversión estratégica que protege el negocio y genera valor a largo plazo.

La Psicología del Testing

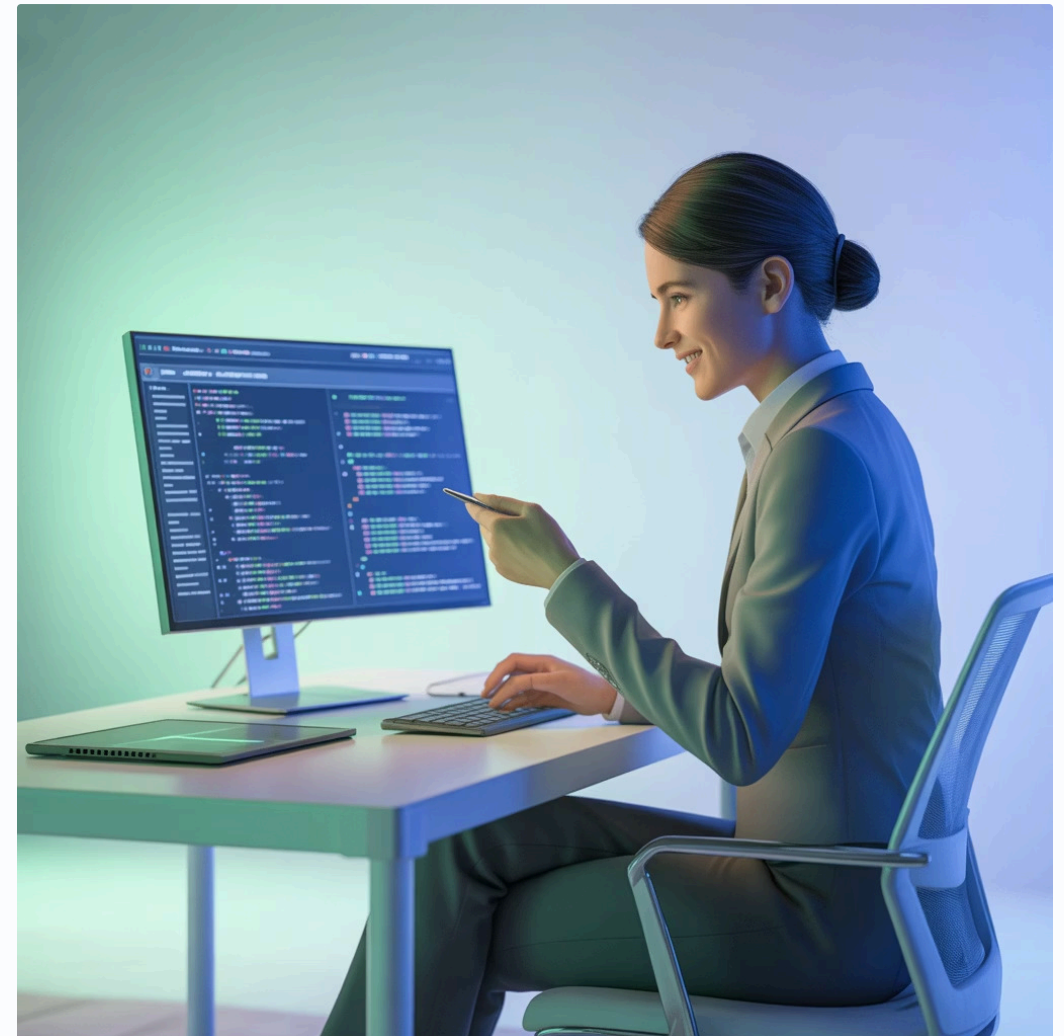
Mentalidad de Desarrollador vs. Mentalidad de Tester



Desarrollador (Constructiva)

El desarrollador construye soluciones y naturalmente busca confirmar que su trabajo funciona correctamente.

- Prueba el "camino feliz" o escenario ideal
- Busca demostrar que el código **funciona**
- Mentalidad optimista y constructiva
- Se enfoca en implementar funcionalidades



Tester (Investigativa)

El tester adopta una perspectiva de investigación crítica y sistemática para descubrir problemas potenciales.

- Tiene "curiosidad disciplinada"
- Busca descubrir dónde el código **no funciona**
- Mentalidad escéptica y analítica
- Prueba casos extremos y escenarios inusuales

Ambas mentalidades son necesarias y complementarias para crear un producto de calidad. La clave está en el balance y la colaboración entre ambos roles.

La pregunta clave: ¿Desde dónde probamos?

El enfoque de testing determina nuestra perspectiva, las herramientas que usamos y el tipo de información que obtenemos. No existe un enfoque "mejor", sino enfoques complementarios que usamos según el contexto.

Pruebas de Caja Negra

(Black-Box Testing)

Sin acceso al código interno, solo vemos entradas y salidas

Pruebas de Caja Blanca

(White-Box Testing)

Con acceso completo al código fuente y la lógica interna

Pruebas de Caja Gris

(Grey-Box Testing)

Enfoque híbrido con conocimiento parcial de la arquitectura

Enfoque 1: Pruebas de Caja Negra (Black-Box)

Probando desde la perspectiva del usuario

Qué es: Probar el sistema como una "caja opaca", sin ver ni entender el código interno. Solo nos importan las entradas que proporcionamos y las salidas que recibimos.

Perspectiva: 100% la del **Usuario Final**. No necesitamos conocimientos técnicos del sistema.

Base de las pruebas: Se basa *únicamente* en los Requisitos funcionales y especificaciones del negocio.

- ❏ **Analogía práctica:** Probar un cajero automático. Ingresas tu tarjeta, digitas el PIN y esperas que salga el dinero correcto. No te importa si internamente usa Java, C++ o Python, ni cómo se comunica con el banco.



Enfoque 2: Pruebas de Caja Blanca (White-Box)

Qué es: Probar el sistema con acceso total al código fuente, la estructura interna, los algoritmos y la lógica de programación. Es el enfoque más técnico.

Perspectiva

100% la del **Desarrollador**. Requiere conocimientos profundos de programación.

Base de las pruebas

Se basa en el Diseño Técnico, arquitectura del software y el código fuente.

Objetivo principal

Verificar la lógica interna, cobertura de código y optimización de algoritmos.

Analogía del mecánico: El mecánico experto que abre el capó del auto y mide el voltaje de cada sensor, verifica las conexiones eléctricas y revisa el funcionamiento interno del motor. No prueba si el auto "anda" desde el punto de vista del conductor, prueba si cada componente interno funciona según especificaciones técnicas.

Enfoque 3: Pruebas de Caja Gris (Grey-Box)

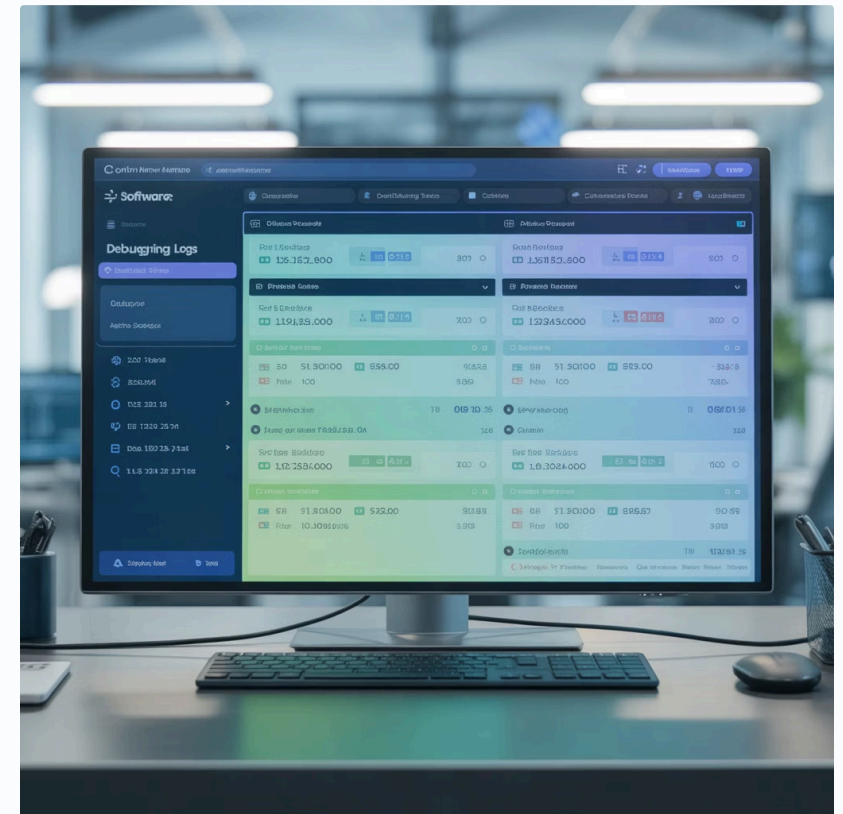
El mejor de ambos mundos

Qué es: Un enfoque híbrido que combina lo mejor de ambos mundos. Realizamos pruebas como un usuario (Caja Negra), pero tenemos conocimiento parcial de la arquitectura interna y acceso a logs, bases de datos y documentación técnica (Caja Blanca).

Ejemplo Práctico: Debugging Avanzado

1. **Prueba inicial:** Como usuario, intentas registrarte en una aplicación web y recibes un mensaje genérico de "Error inesperado"
2. **Análisis con Caja Gris:** A diferencia de un usuario común, tú puedes acceder a los logs del servidor, revisar la base de datos y analizar las llamadas API
3. **Diagnóstico preciso:** Descubres que "El usuario Sí se creó correctamente en la base de datos, pero el problema es que el backend está devolviendo un código de error 500 al frontend en lugar de un 200, generando el mensaje confuso"

Este nivel de detalle solo es posible con acceso parcial a la arquitectura.



¿Cómo diseñar Pruebas de Caja Negra?

No podemos probar "al azar"

Si un campo de entrada acepta edades de 1 a 100, ¿vamos a ejecutar 100 pruebas diferentes? Absolutamente no. Eso sería ineficiente, costoso e impracticable. En su lugar, usamos **técnicas de diseño de pruebas** para elegir estratégicamente los casos más efectivos y eficientes.

Veremos 3 técnicas fundamentales:

1. Partición de Equivalencia

(Equivalence Partitioning - EP)

2. Análisis de Valores Límite

(Boundary Value Analysis - BVA)

3. Tablas de Decisión

(Decision Tables)



Técnica 1: Partición de Equivalencia (EP)

El principio de agrupación inteligente

Definición: Dividimos todos los datos de entrada posibles en "clases de equivalencia" o particiones, donde todos los valores dentro de una clase deberían comportarse de manera idéntica. Luego probamos *un solo valor representativo* de cada clase.

Esta técnica reduce dramáticamente el número de pruebas necesarias sin sacrificar la efectividad.



Ejemplo Práctico: Campo "Edad" (rango válido: 18-65 años)

1	2
Partición Inválida: Edad < 18 Probamos con 10 años. Cualquier valor menor a 18 debería comportarse igual.	Partición Válida: Edad 18-65 Probamos con 35 años. Cualquier valor en este rango es válido.
3	4
Partición Inválida: Edad > 65 Probamos con 70 años. Cualquier valor mayor a 65 debería rechazarse.	Partición Inválida: No numérico Probamos con "ABC" . Datos que no son números deben ser rechazados.

☐ **El poder de EP:** Con apenas **4 pruebas estratégicas**, acabamos de cubrir efectivamente millones de posibilidades de entrada. Esta es la esencia del testing inteligente.

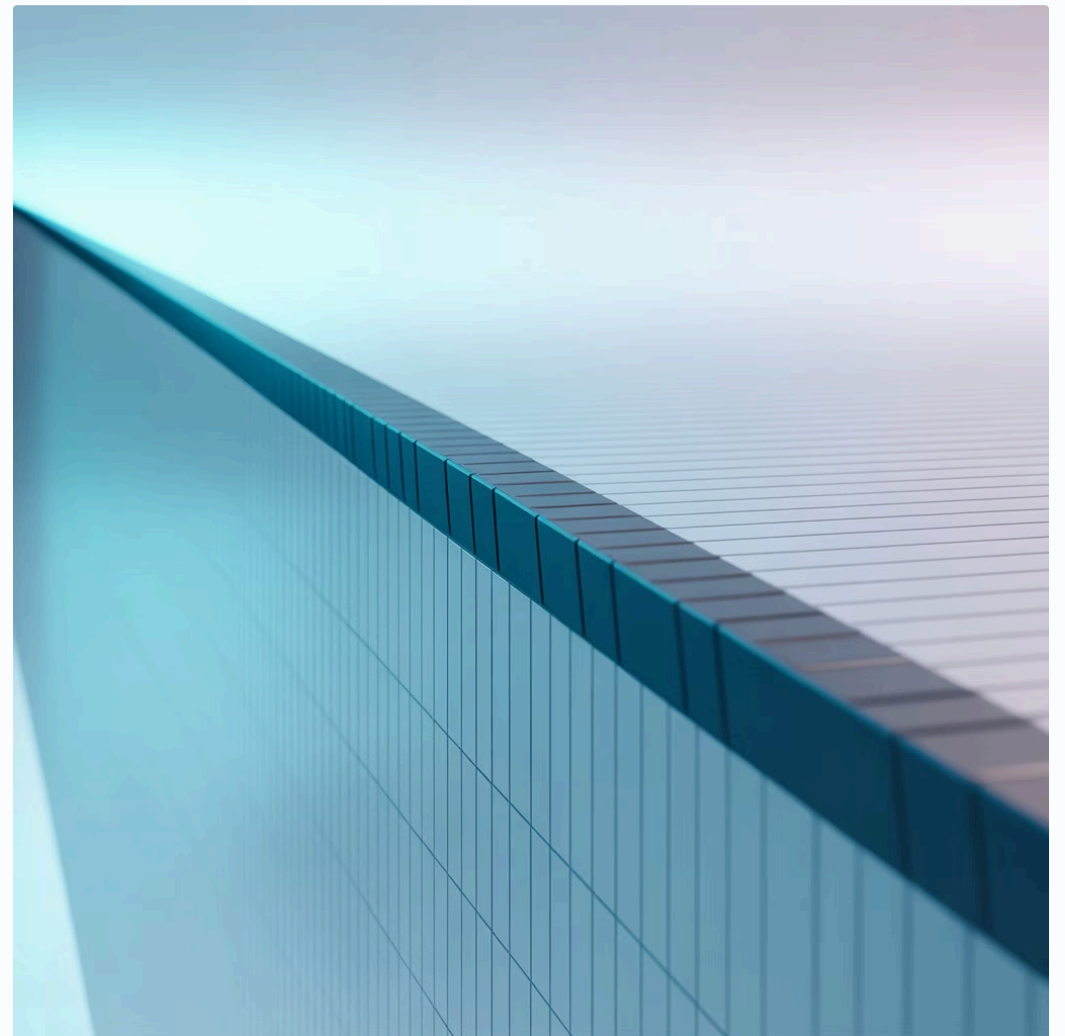
Técnica 2: Análisis de Valores Límite (BVA)

La técnica más importante del testing

Definición: Esta técnica se basa en una verdad fundamental del desarrollo de software: los errores *casi siempre* ocurren en los "bordes" o límites de las condiciones. Un simple error de tipeo (como usar > en lugar de >=) causa comportamientos inesperados justo en los límites.

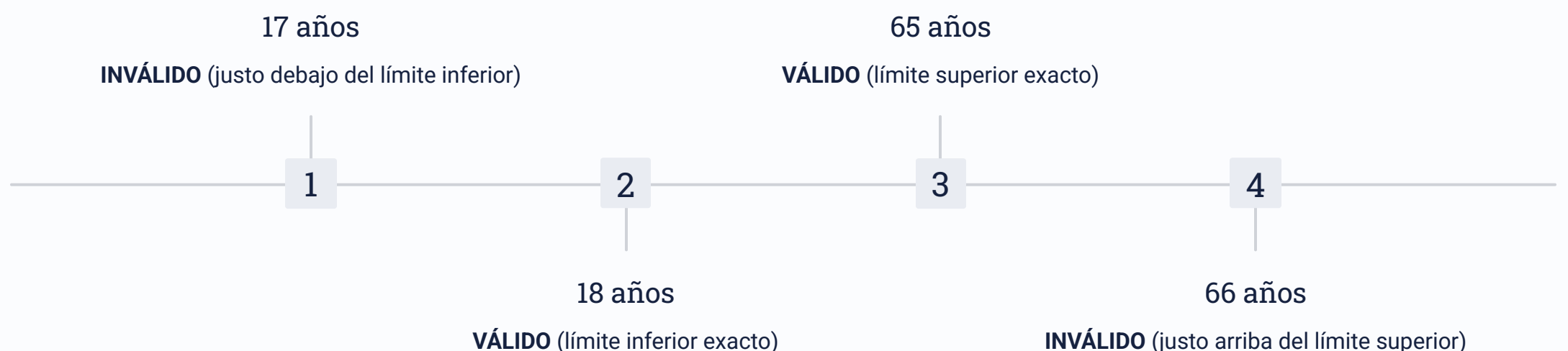
¿Por qué los límites son tan críticos?

- Los desarrolladores cometen errores en operadores de comparación
- Las condiciones off-by-one son extremadamente comunes
- Los límites revelan problemas de lógica condicional



Ejemplo: Campo "Edad" (rango válido: 18-65 años)

Probamos los límites exactos y sus vecinos inmediatos:



BVA es donde encontramos la mayoría de los bugs. Dominar esta técnica es esencial para cualquier tester profesional.

Técnica 3: Tablas de Decisión

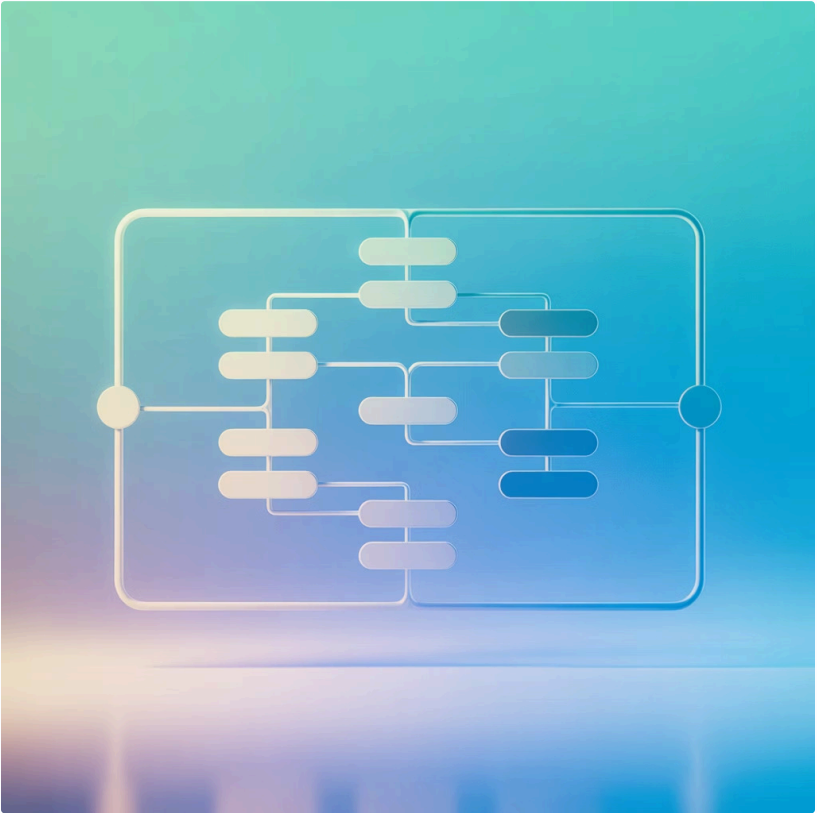
Definición: Esta técnica se usa para probar **reglas de negocio complejas** que dependen de **MÚLTIPLES** condiciones combinadas. Cuando tenemos varios factores que interactúan, necesitamos una forma sistemática de probar todas las combinaciones posibles.

Ejemplo de Negocio: Sistema de Descuentos

Regla: "Se aplica un descuento del 15% SI y SOLO SI el cliente es 'Premium' Y la compra supera \$50.000"

Con 2 condiciones booleanas (Sí/No), esto genera **4 combinaciones posibles** (2×2) que debemos probar exhaustivamente:

Regla	¿Cliente Premium?	¿Compra > \$50k?	¿Aplica Descuento?
1	✓ Sí	✓ Sí	SÍ - 15%
2	✓ Sí	✗ No	NO
3	✗ No	✓ Sí	NO
4	✗ No	✗ No	NO



Las Tablas de Decisión garantizan que no olvidemos ninguna combinación crítica en reglas de negocio complejas.

¿Cómo diseñar Pruebas de Caja Blanca?

El Objetivo Central: Cobertura de Código (Code Coverage)

Definición: La cobertura de código es una métrica expresada en porcentaje (%) que nos indica cuánto de nuestro código fuente fue realmente "ejecutado" o "tocado" por nuestras pruebas automatizadas.

Cobertura de Sentencia

(Statement Coverage)

¿Se ejecutó cada línea de código al menos una vez durante las pruebas?
Es el nivel básico de cobertura.

Cobertura de Decisión

(Decision/Branch Coverage)

Más importante: ¿Probamos todos los caminos posibles de cada IF / ELSE, cada switch, cada condición?

Ejemplo crítico de Cobertura de Decisión

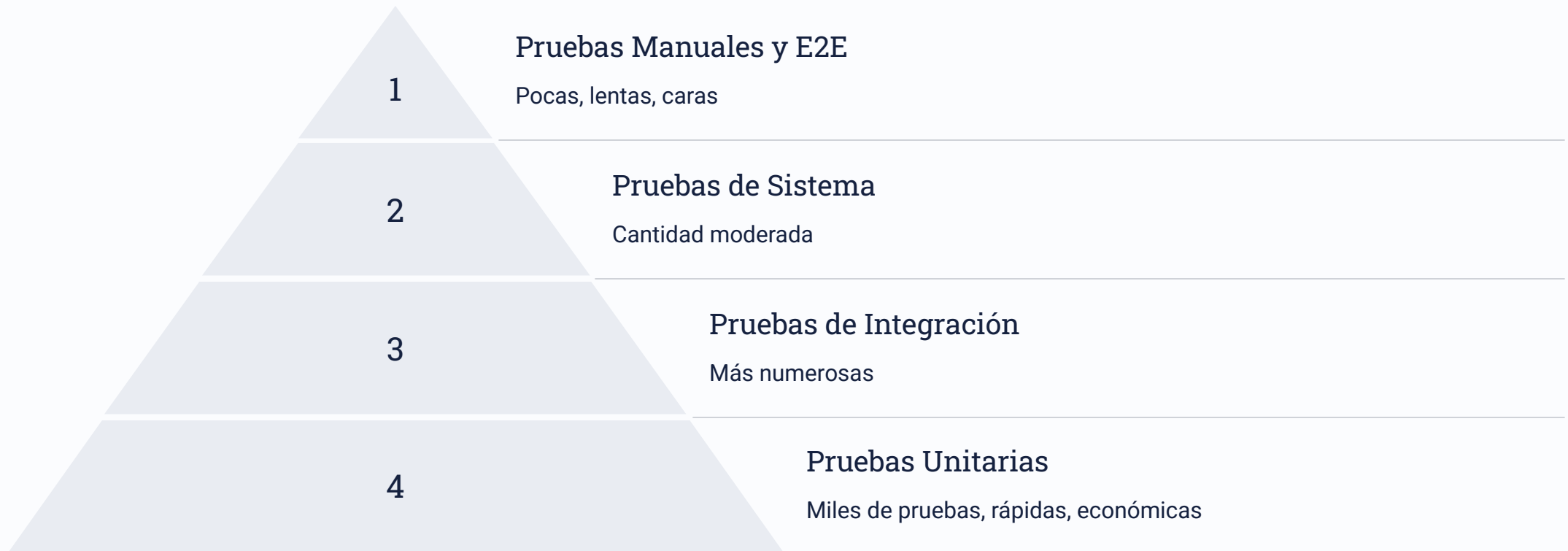
```
if (edad >= 18) {  
    // Camino TRUE - permitir acceso  
    return "Acceso permitido";  
} else {  
    // Camino FALSE - denegar acceso  
    return "Acceso denegado";  
}
```

Si solo probamos con edad = 25, ejecutamos el camino TRUE y obtenemos **50% de cobertura de decisión**. ¡Debemos también probar con edad = 15 para ejecutar el camino FALSE y alcanzar el 100%!

Los Niveles de Prueba: ¿Cuándo probamos?

La Pirámide del Testing

La Pirámide del Testing es un modelo visual que nos guía sobre *cuánto* esfuerzo, tiempo y recursos dedicar a cada nivel de pruebas. No todos los tipos de pruebas son iguales en costo ni velocidad.



Principio clave: La base (pruebas unitarias) debe ser sólida y amplia. A medida que subimos, las pruebas son más costosas pero también más realistas.

Nivel 1 (Base): Pruebas Unitarias (Unit Tests)

La fundación de la calidad del software

Qué es: Pruebas automatizadas que verifican la pieza más pequeña y atómica de código (típicamente una función o método) de forma completamente **aislada** del resto del sistema.

Enfoque: 100% Caja Blanca. Requiere acceso al código fuente.

Quién las escribe: El propio Desarrollador, idealmente antes o durante el desarrollo (TDD - Test Driven Development).



Ejemplo Práctico

Imagina una función simple que calcula el IVA (21%) sobre un monto:

```
function calcularIVA(monto) {  
  return monto * 0.21;  
}
```

La prueba unitaria verifica: `calcularIVA(100) === 21`

1000+

Cantidad típica

Un proyecto puede tener miles de pruebas unitarias

< 1s

Velocidad

Ejecutan en milisegundos

\$

Costo

Muy baratas de mantener

Nivel 2: Pruebas de Integración

Verificando que las piezas trabajan juntas

Qué es: Pruebas que verifican que dos o más módulos, componentes o servicios funcionan correctamente **juntos** cuando se conectan. Ya no probamos en aislamiento sino la comunicación entre partes.

Enfoque: Caja Gris. Conocemos la arquitectura pero probamos las interfaces.

Ejemplo Típico

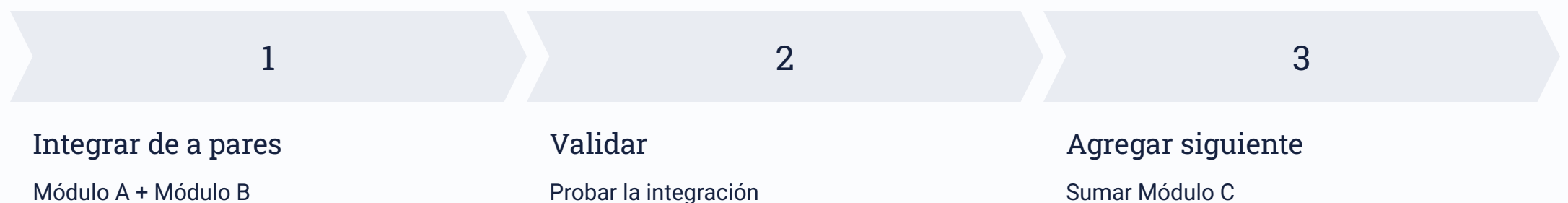
Probar que tu API (backend) puede:

- Conectarse exitosamente a la Base de Datos
- Escribir un nuevo registro de usuario
- Leer ese registro correctamente
- Actualizar y eliminar datos



Estrategia: Evitar el "Big Bang"

No integramos todos los módulos al final en un gran evento caótico. En su lugar, usamos **integración incremental**:



Este enfoque reduce drásticamente la complejidad y facilita identificar dónde ocurren los problemas de integración.

Nivel 3: Pruebas de Sistema (End-to-End)

Qué es: Pruebas que evalúan el sistema **completo y totalmente integrado**, tal como lo experimentaría un usuario real en producción. Es el nivel más cercano a la realidad.

Enfoque: 100% Caja Negra. No nos importa cómo funciona internamente, solo importa si funciona desde la perspectiva del usuario.

Las pruebas de sistema se dividen en dos grandes categorías:

Pruebas Funcionales

¿Qué? Verifican que las funcionalidades hacen lo que deben hacer.

Ejemplos:

- ¿El botón de "Comprar" agrega el producto al carrito?
- ¿Se procesa correctamente el pago?
- ¿Llega el email de confirmación?
- ¿Se actualiza el inventario?

Pruebas No Funcionales

¿Cómo? Verifican la calidad de la experiencia.

Ejemplos:

- **Performance:** ¿El botón responde en < 2 segundos?
- **Seguridad:** ¿Los datos están encriptados?
- **Usabilidad:** ¿Es intuitivo y fácil de usar?
- **Accesibilidad:** ¿Funciona con lectores de pantalla?

Nivel 4 (Punta): Pruebas de Aceptación (UAT)

User Acceptance Testing

Qué es: El nivel final y más crítico. Son pruebas que confirman que el software cumple con los requisitos del **negocio** y está realmente listo para que el cliente lo "acepte" y lo use en producción.

Enfoque: 100% Caja Negra desde la perspectiva del negocio, no técnica.

Quién las ejecuta: El Cliente final, usuarios reales o representantes del negocio (Product Owners).

Objetivo: Responder la pregunta definitiva: "¿Esto resuelve nuestro problema de negocio?"



Tipos de Pruebas de Aceptación



Pruebas Alpha (α)

Las ejecutan empleados de tu propia empresa pero de otros departamentos (marketing, ventas, etc). Se hacen en un ambiente controlado interno antes de liberar externamente.



Pruebas Beta (β)

Se libera el software a un grupo selecto de usuarios reales externos (early adopters) antes del lanzamiento oficial. Permiten detectar problemas en condiciones reales de uso.

Conclusión y Puente a la Unidad 8

¡Ahora ya tenés el "mapa" completo del testing!

Hoy (Unidad 7) vimos la TEORÍA

Construimos los fundamentos conceptuales que todo tester profesional debe dominar:

Enfoques de Testing

Caja Negra, Blanca y Gris - desde dónde probamos

Niveles de Prueba

Unitarias, Integración, Sistema y Aceptación - cuándo probamos

Técnicas de Diseño

EP, BVA, Tablas de Decisión, Cobertura - cómo probamos inteligentemente

Próxima Clase (Unidad 8) veremos la PRÁCTICA

Transformaremos la teoría en habilidades aplicables en el mundo real:

01

Documentación

¿Cómo se documenta profesionalmente un Caso de Prueba completo?

02

Gestión de Defectos

¿Cómo se reporta un bug con Severidad Crítica/Alta efectivamente?

03

Gestión del Testing

¿Cómo se planifican y ejecutan Pruebas de Regresión?

📌 **Preparación para la próxima clase:** Revisa los conceptos de hoy, especialmente las técnicas de diseño (EP y BVA), ya que las aplicaremos en ejercicios prácticos reales.